Toward Reproducible EMG-Based Driving Control: Fixed Sensor Placement, CNN Inference, and CARLA-Based Evaluation

[Author names and affiliations]

April 7, 2026

Abstract

This draft presents the current capstone continuation of an EMG-based driving interface built around Delsys Trigno sensing, a fixed sensor-placement workflow, convolutional neural network gesture classification, dual-arm realtime inference, and CARLA-based downstream evaluation. The maintained implementation collects four-gesture EMG data through a custom Python GUI, resamples heterogeneous sensor streams to a common 2000 Hz time grid, filters the EMG signal with fixed notch and bandpass filters, and trains `GestureCNNv2`, a residual 1D CNN with channel attention and an explicit energy bypass to preserve rest-versus-active amplitude cues. The deployed realtime path is fixed to the four-gesture set `neutral`, `left_turn`, `right_turn`, and `horn`, and publishes fused split-or-single dual-arm outputs for CARLA control. Evaluation is organized around three main result groups: offline classification performance, end-to-end latency, and CARLA scenario performance. Relative to prior driving-focused EMG studies, especially Basnet et al., the present work shifts emphasis toward a stricter sensing contract, a more reproducible model/runtime pipeline, and a report-ready evaluation stack. Final numerical results will be inserted once the experiment set is frozen.

1 Introduction

EMG-based human–machine interfaces remain attractive for driving-related control because they offer a potential pathway for non-traditional or accessibility-oriented interaction without relying entirely on conventional manual input. In practice, however, an EMG driving interface is only useful if it is stable enough to move beyond isolated offline classification and into reproducible runtime behavior. This makes system discipline as important as raw classifier accuracy.

The current work continues earlier EMG-for-driving research by tightening the full experimental contract rather than simply increasing the number of gesture classes. Three engineering issues shaped the present design. First, inconsistent sensor placement made cross-session channel meaning unstable. Second, heterogeneous Delsys sensor sampling rates required explicit preprocessing before training and deployment. Third, the control layer needed a narrow, auditable gesture-to-vehicle contract so that downstream simulator behavior could be evaluated credibly rather than anecdotally.

This paper therefore centers on a narrower and more defensible story: a fixed-layout four-gesture EMG pipeline for driving-related control. The main contributions of the current system are:

- a fixed pair-number-specific sensor-placement workflow for both training and inference;
- a preprocessing path that resamples multi-sensor EMG to a common time base and applies a mirrored offline/realtime filtering stack;
- a residual 1D CNN with channel attention and an explicit pre-normalization energy bypass for improved neutral-versus-active separation;

- a dual-arm deployment path that publishes fused split-or-single gesture outputs under a fixed four-gesture contract; and
- a CARLA evaluation harness with logged latency and scenario-level driving metrics.

At the time of writing, the maintained repo state is fixed to the gesture set `neutral`, `left_turn`, `right_turn`, and `horn`. Broader gesture vocabularies remain possible in historical or experimental branches, but they are not part of the active training, realtime, and CARLA evaluation path described in this draft.

2 Related Work

Broader myoelectric-interface literature has long emphasized that electrode placement, segmentation strategy, preprocessing, and classifier choice strongly affect downstream performance and reproducibility [1, 2]. That observation is especially relevant here because driving-related control is less tolerant of inconsistency than a standalone gesture-recognition benchmark.

On the gesture-recognition side, many published studies focus on maximizing offline classification accuracy on constrained hand-gesture datasets. Examples include compact CNN-based and multi-stream deep-learning approaches for sEMG recognition [3, 4]. These works are useful for understanding model design, but they typically do not evaluate whether the resulting predictions remain usable when inserted into a closed-loop vehicle-control task.

Driving-specific EMG interfaces are far less common. Gomez-Gil et al. explored EMG-based steering for a tractor [5]. Nacpil et al. developed and evaluated sEMG-controlled steering-assistance interfaces for simulator and low-speed automobile settings [6, 7]. De Freitas et al. presented a proof-of-concept EMG-controlled car using low-cost hardware and machine learning [8]. These studies demonstrate the feasibility of EMG-mediated steering or vehicle control, but they generally focus on proof-of-concept steering assistance, limited maneuver sets, or hardware/interface validation rather than a fully documented acquisition-to-evaluation pipeline.

The closest prior work to the present study is Basnet et al. [9]. Their paper evaluated an EMG-based human–machine interface for driving with 19 able-bodied participants, a dominant-forearm Delsys setup, three movement categories, 50 ms sliding windows, time-domain features, and an LDA classifier. They examined straight lane driving, overtaking, and 90-degree turns at intersections while also measuring blink-rate workload, usability, and safety. The present work should be framed as a continuation of that research direction, but with a different emphasis: rather than broadening the human-factors instrumentation, the capstone branch concentrates on reproducibility of the sensing contract, explicit training/runtime alignment, and a CARLA evaluation stack that can be rerun and audited from saved artifacts.

In concise terms, the literature can be divided into three comparison groups. First, general myoelectric-interface and gesture-recognition papers describe signal-processing and modeling techniques but usually stop at offline recognition quality [1, 2, 3, 4]. Second, steering-specific EMG papers demonstrate that EMG can influence vehicle or steering control, but often in narrow proof-of-concept settings [5, 6, 7, 8]. Third, Basnet et al. provide the nearest full driving-task comparison by measuring performance, workload, usability, and safety in a simulator setting [9]. The present draft is positioned closest to that third group, but with a more explicit emphasis on reproducible data flow, fixed sensor placement, dual-arm CNN deployment, and scriptable post-run analysis.

Insert system pipeline figure here: collection → preprocessing → training → realtime inference → CARLA evaluation.

Figure 1: End-to-end system overview. Replace this placeholder with a final architecture diagram that shows the maintained data and control flow.

3 System Overview

The active system pipeline is collection → resampling → filtering → windowing and label generation → CNN training → dual-arm realtime inference → CARLA control evaluation. Data collection is performed with a custom Delsys GUI, preprocessing and training are implemented in Python and PyTorch, and downstream control experiments are conducted in CARLA through a simulator client that launches realtime inference internally.

The active top-level entrypoints are `DelsysPythonGUI.py` for collection, `emg/resample_raw_dataset.py` and `emg/filtering.py` for preprocessing, `train_per_subject.py` and `train_cross_subject.py` for model training, `realtime_gesture_cnn.py` for standalone inference, and `carla_integration/manual_control_emg.py` for simulator runs. Strict raw collections are stored under `data_strict/`, resampled and filtered strict data under `data_resampled_strict/`, and trained bundles under `models/strict/`.

4 Methods

4.1 Data acquisition

EMG acquisition is performed with Delsys Trigno hardware through a custom Python GUI that supports explicit sensor pairing, scanning of paired devices, live EMG viewing, and scripted labeled collection protocols. The maintained protocol for the active workflow is `standard_4g`, which records the fixed trainable label set `left_turn`, `right_turn`, `neutral`, and `horn`. Each prompted gesture lasts 5.0 s, neutral also lasts 5.0 s, five repetitions are recorded, and calibration is enabled with 5.0 s neutral-rest capture, 5.0 s MVC capture, and a 2.0 s MVC preparation interval.

Between prompted gestures, the collection flow inserts a 1.5 s neutral rest period labeled `neutral_buffer`. The first 0.5 s of a prompted label is trimmed so that onset transitions are not treated as settled gesture examples, and `neutral_buffer` is retained in the raw file only as a collection artifact rather than a trainable class. The GUI still contains several focus and diagnostic protocols for targeted recollection, but the maintained training and deployment path described in this paper is fixed to the four-gesture contract above.

Raw collections are saved as NPZ files containing the raw EMG matrix, per-channel timestamps, per-sample labels, the protocol event list, session metadata, channel-label metadata, and optional

Insert gesture image: `neutral`

Insert gesture image: `left_turn`

Insert gesture image: `right_turn`

Insert gesture image: `horn`

Figure 2: Maintained four-gesture set used by the current pipeline. Replace each placeholder with the final gesture photograph or illustration used in the paper.

calibration arrays. That metadata is critical because later training and realtime stages resolve channels from the saved pair identities rather than trusting scan order alone.

Figure 2 is the intended insertion point for the four maintained gesture illustrations used throughout the current branch. Replacing those placeholders with standardized still images will make the final paper easier to read and will anchor the gesture vocabulary visually before the methods and results sections refer back to it.

4.2 Strict sensor-placement policy

The fixed sensor-placement workflow is central to the current method. Earlier mixed-layout assumptions were replaced because they allowed the semantic meaning of channel indices to drift between sessions. The active helper resolves channels by fixed pair identity and fails closed if required pair numbers are missing, duplicated, or inconsistent with the arm-specific layout contract.

For the right arm, pairs 1, 2, and 3 are assigned to the three Avanti slots, pair 7 to Maize, pair 9 to Galileo, and pair 11 to Mini. For the left arm, pairs 4, 5, and 6 are assigned to the three Avanti slots, pair 8 to Maize, and pair 10 to Galileo. This yields 17 strict channels on the right arm and 16 on the left. Channels are reordered by fixed pair identity during both training and inference.

Figure 3 is the intended placement for the left-arm and right-arm sensor-placement figures. Those images should be included in the final manuscript because the strict-layout claim is central to the methodological contribution of this branch.

4.3 Preprocessing

Different Delsys sensor types can stream at different effective sample rates, so raw multi-sensor sessions are first resampled to a common time base. The active resampler estimates per-channel sample rates from timestamp differences, finds the overlapping valid time interval across channels, constructs a uniform 2000 Hz grid, linearly interpolates each channel onto that grid, and transfers labels to the new grid by nearest-neighbor remapping. Calibration arrays are resampled onto the same time base so later normalization still reflects the maintained preprocessing contract.

Insert right-arm sensor-placement image

Insert left-arm sensor-placement image

Figure 3: Fixed sensor-placement contract for the maintained workflow. Replace the placeholders with the final right-arm and left-arm placement images, labeled by pair number and sensor type where helpful.

After resampling, the active filtering stack applies a 60 Hz notch, a 120 Hz notch, and a 20–450 Hz sixth-order bandpass filter. The same filter family is mirrored in the realtime path so that deployment does not operate on a materially different signal than the one used during training.

4.4 Windowing and label generation

The filtered EMG stream is segmented into overlapping 200-sample windows with a 100-sample step. At the 2000 Hz resampled rate, this corresponds to 100 ms windows with 50 ms stride. Each window is labeled by the majority class of its constituent samples. The inter-gesture rest label `neutral_buffer` is retained during collection but excluded from model training. Windows with insufficient label purity can also be dropped through a configurable minimum label-confidence threshold.

4.5 Model architecture

The active classifier is `GestureCNNv2`, which consumes EMG windows shaped as $(\text{channels}, \text{time})$. The network applies input `InstanceNorm1d` followed by a residual 1D convolutional backbone with three stages at 32, 64, and 128 channels. Each stage includes squeeze-and-excitation channel attention, and the final embedding is global-average pooled before classification.

A key design feature is an explicit pre-normalization energy bypass. Before input normalization, the model computes the mean squared energy of the raw window and concatenates this scalar to the learned representation before the final linear classifier. The purpose of this bypass is to preserve amplitude information that would otherwise be attenuated by per-window normalization, particularly for separating rest-like neutral windows from active gesture windows.

4.6 Training configuration

The repository maintains both per-subject and cross-subject training paths. The per-subject configuration currently uses 40 epochs per fold, batch size 512, Adam optimization with learning rate 10^{-4}, dropout 0.25, label smoothing 0.05, calibration-aware normalization when calibration quality passes threshold checks, and a minimum label-confidence threshold of 0.85. Per-subject evaluation uses session-grouped `StratifiedGroupKFold` so windows from the same session file do not leak across train and validation folds. After cross-validated evaluation, one final per-subject model is fit on all windows using an epoch count chosen from the fold-level best epochs.

The cross-subject path uses the same general architecture but trains for 50 epochs, applies a wider augmentation amplitude range to reflect greater inter-subject variation, and uses subject-

Insert driving-rig photograph here: participant setup, workstation, sensors, and simulator display.

Figure 4: Driving rig used for collection and simulator evaluation. Replace this placeholder with the final photograph of the experimental setup.

balanced sampling via `WeightedRandomSampler`. It also runs leave-one-subject-out evaluation before fitting the final pooled model. This distinction is important because subject-specific and pooled models answer different questions about personalization and generalization.

4.7 Deployment and runtime post-processing

The current deployment path uses `realtime_gesture_cnn.py` in dual-arm mode and is fixed to the four-gesture subset {`neutral`, `left_turn`, `right_turn`, `horn`}. Incoming streams are resampled to 2000 Hz, filtered using the same notch and bandpass stack as the offline path, and optionally normalized using live neutral and MVC calibration terms. If the MVC-to-neutral quality ratio is too weak for a given arm, normalization is disabled for that arm instead of applying unreliable scaling.

The current runtime tuning values are defined directly in `emg/runtime_tuning.py`. In the checked-in maintained state, the realtime preset uses smoothing of 1 frame, a minimum confidence threshold of 0.80, a dual-arm agreement threshold of 0.55, no output hysteresis, and no softmax-margin rejection. Each arm is decoded independently first, and the two arm outputs are then fused into a published `PublishedGestureOutput` that can be either `single` or `split`. This published interface is the downstream contract consumed by CARLA.

5 Experimental Setup

5.1 Hardware and software

The active software environment targets Python 3.10 with NumPy, SciPy, scikit-learn, libemg, PyTorch, PySide6, pygame, and the CARLA Python API. Live data collection expects Windows with the Delsys SDK bridge through `pythonnet`. The simulator client is implemented in CARLA through a modified manual-control interface that launches realtime inference internally.

5.2 Dataset scope

The narrowest defensible reported configuration on the current branch is the strict-layout four-gesture setting. In that configuration, training and deployment are both restricted to `neutral`,

`left_turn`, `right_turn`, and `horn`. The final manuscript should explicitly state which subjects, arms, sessions, and strict dataset roots were used for the reported experiments.

5.3 CARLA scenarios

To support repeatable downstream evaluation, the simulator layer includes two maintained scenario presets: `lane_keep_5min` and `highway_overtake`. Both currently run on `Town04_Opt`, use a fixed sedan blueprint, and write per-tick state to CSV logs, including checkpoint progress and completion state. The lane-keeping scenario starts timing once the ego vehicle crosses the first active checkpoint, while the overtake scenario requires both a successful pass-and-return maneuver and crossing the finish checkpoint.

The maintained gesture-to-control contract is intentionally narrow. The left arm issues strong steering requests, the right arm issues weaker steering requests, no gesture directly applies throttle or brake, and a dual-arm `horn` command toggles reverse on or off. The final paper should report the exact frozen scenario configuration used for the final results and make clear that CARLA consumes the published dual-arm output from the realtime pipeline rather than raw per-window logits.

6 Evaluation Protocol

The main reported metric set for this paper should match the capstone report. Prompted realtime-behavior metrics can still be used as supplementary analysis, but the core evaluation should remain centered on offline model quality, end-to-end latency, and downstream simulator behavior.

6.1 Offline metrics

The maintained offline metrics are balanced accuracy, macro precision, macro recall, macro F1, and worst-class recall. Confusion matrices and classwise recall can be added as supporting analysis, but these five metrics should remain the headline offline results because they are the metrics harvested directly and consistently from the maintained model bundles.

6.2 Latency metrics

Latency analysis joins realtime prediction logs and CARLA drive logs on a shared prediction identifier. The maintained headline latency outputs are mean end-to-end latency and p95 end-to-end latency, where end-to-end latency is measured from `window_end_ts` in the realtime log to `control_apply_ts` in the CARLA log.

6.3 CARLA scenario metrics

The maintained CARLA metrics are scenario success, completion time, mean velocity, lane offset mean, steering angle mean, steering entropy, lane error RMSE, and lane invasions. Optional diagnostics such as `mean_velocity_deviation_mps` or `command_success_rate` can be included if the underlying logs support them, but they are not part of the primary metric set that this draft is organized around.

Insert offline model summary plot here, for example the output of `plot_model_summary.py`.

Figure 5: Offline model-summary figure placeholder. Replace this with the final plot comparing the selected offline metrics across the reported conditions.

Insert latency summary plot here, for example the output of `plot_latency_summary.py`.

Figure 6: Latency-summary figure placeholder. Replace this with the final end-to-end latency figure used in the paper.

7 Results

7.1 Offline classification results

[Insert the main offline results table here. The expected headline metrics are balanced accuracy, macro precision, macro recall, macro F1, and worst-class recall for the selected per-subject and/or cross-subject bundles.]

7.2 Latency results

[Insert the latency summary table here. The maintained headline metrics are mean end-to-end latency and p95 end-to-end latency, computed from paired realtime and CARLA logs.]

7.3 CARLA scenario results

[Insert the lane-keeping and overtake scenario table here. Include scenario success, completion time, mean velocity, lane offset mean, steering angle mean, steering entropy, lane error RMSE, and lane invasions.]

Insert CARLA metrics summary plot here, for example the output of `plot_carla_summary_bars.py`.

Figure 7: CARLA scenario-summary figure placeholder. Replace this with the final CARLA metrics figure comparing the reported driving scenarios or reported conditions.

7.4 Comparison to prior work

The strongest direct comparison target is Basnet et al. [9]. That comparison should be limited to metrics and scenarios that are genuinely comparable. For example, lane-position error, steering-angle behavior, steering entropy, and maneuver-completion or success outcomes can be compared at a methodological level, while blink-rate workload, USE usability scores, and safety-composite scores should not be treated as directly comparable because they are not part of the maintained capstone evaluation stack.

Comparison to other published EMG-control papers should also be bounded carefully. Earlier steering-assistance and proof-of-concept papers [5, 6, 7, 8] are best used to position the novelty of the present pipeline rather than to claim numerical superiority. Their sensing setups, actuator/control mappings, vehicles, and maneuver sets differ substantially. In contrast, the more general sEMG recognition papers [3, 4] are best used to justify model-design choices and to explain why high offline recognition performance alone is not sufficient for a driving-control argument.

8 Discussion

The main technical discussion should focus on whether stricter data discipline, a fixed four-gesture contract, and a narrower control interface materially improved system credibility. In particular, the current branch suggests that layout control, preprocessing consistency, and explicit publication and control contracts are at least as important to deployment quality as the underlying network architecture.

One recurring design tradeoff is between control breadth and reliability. The repository contains targeted collection utilities and both per-subject and cross-subject training paths, but the maintained deployment story is intentionally narrow so that the end-to-end system can be evaluated defensibly. Another is the tradeoff between flexible realtime behavior and downstream interpretability: by freezing gesture labels, scenario presets, and shared metrics, the system becomes easier to audit even if broader behaviors remain out of scope.

This framing also clarifies the relationship to Basnet et al. [9]. Basnet's study is stronger as a human-factors comparison across control conditions, whereas the present capstone branch is stronger as a systems paper about reproducibility, deployment alignment, and scriptable evaluation. Those are complementary rather than contradictory contributions, and the final manuscript should

state that plainly.

The final paper should also state the current limitations plainly. These may include limited subject count, the focus on simulator evaluation rather than real vehicles, the requirement for consistent fixed sensor placement, and any scenario configurations that were still being tuned when experiments were collected.

9 Conclusion

This draft paper documents a fixed-layout EMG control pipeline that connects Delsys-based collection, a reproducible preprocessing workflow, a residual 1D CNN with an energy bypass, dual-arm realtime inference on a fixed four-gesture set, and CARLA-based downstream evaluation. The clearest current contribution is not merely a working classifier, but a more defensible experimental system whose sensing contract, deployment behavior, and evaluation outputs are explicitly controlled.

The next step is to freeze the final experiment set, insert the corresponding offline, latency, and CARLA results, and then tighten the claims so that every reported conclusion maps directly to collected evidence.

Acknowledgments

[Insert sponsor, supervisor, lab, and support acknowledgments here.]

References

[1] M. Hakonen, H. Piitulainen, and A. Visala. Current state of digital signal processing in myoelectric interfaces and related applications. *Biomedical Signal Processing and Control*, 18:334–359, 2015. doi:10.1016/j.bspc.2015.02.009.

[2] A. Jaramillo-Yánez, M. E. Benalcázar, and E. Mena-Maldonado. Real-time hand gesture recognition using surface electromyography and machine learning: A systematic literature review. *Sensors*, 20(9):2467, 2020. doi:10.3390/s20092467.

[3] L. Chen, J. Fu, Y. Wang, H. Li, and B. Zheng. Hand gesture recognition using compact CNN via surface electromyography signals. *Sensors*, 20(3):672, 2020. doi:10.3390/s20030672.

[4] J. Shin, A. S. M. Miah, S. Konnai, I. Takahashi, and K. Hirooka. Hand gesture recognition using sEMG signals with a multi-stream time-varying feature enhancement approach. *Scientific Reports*, 14(1):22061, 2024. doi:10.1038/s41598-024-72996-7.

[5] J. Gomez-Gil, I. San-Jose-Gonzalez, L. F. Nicolas-Alonso, and S. Alonso-Garcia. Steering a tractor by means of an EMG-based human-machine interface. *Sensors*, 11(7):7110–7126, 2011. doi:10.3390/s110707110.

[6] E. J. C. Nacpil, Z. Wang, R. Zheng, T. Kaizuka, and K. Nakano. Design and evaluation of a surface electromyography-controlled steering assistance interface. *Sensors*, 19(6):1308, 2019. doi:10.3390/s19061308.

[7] E. J. C. Nacpil and K. Nakano. Surface electromyography-controlled automobile steering assistance. *Sensors*, 20(3):809, 2020. doi:10.3390/s20030809.

[8] R. C. de Freitas, R. Alves, A. G. da Silva Filho, R. E. de Souza, B. L. D. Bezerra, and W. P. dos Santos. Electromyography-controlled car: A proof of concept based on surface electromyography, extreme learning machines and low-cost open hardware. *Computers & Electrical Engineering*, 73:167–179, 2019. doi:10.1016/j.compeleceng.2018.11.012.

[9] N. Basnet, S. Allahvirdi, C. Nadri, J. Park, and M. Zahabi. Evaluating the feasibility of EMG-based human-machine interfaces for driving. *Human Factors*, 68(1):123–141, 2026. doi:10.1177/00187208251367179.